

CloudNative-App-Deployment

CloudNative-App-Deployment is an end-to-end cloud-native CI/CD project that containerizes the **Trendify** application with Docker, automates build and deployment using Jenkins, provisions AWS infrastructure using Terraform, deploys the application on **Amazon EKS** using Kubernetes, exposes it through an **AWS Load Balancer**, and monitors the cluster using **Prometheus**, **Grafana**, and **Metrics Server**.

Table of Contents

- Overview
 - Architecture
 - Tech Stack
 - Project Structure
 - Prerequisites
 - Setup & Deployment Guide
 - 1. Clone the Repository
 - 2. Containerize the Application
 - 3. Provision AWS Infrastructure with Terraform
 - 4. Configure Jenkins
 - 5. Deploy to Amazon EKS
 - 6. CI/CD Pipeline
 - 7. Monitoring
 - Accessing the Application
 - Accessing Monitoring Dashboards
 - Cleanup
 - Project State
 - Screenshots
 - Author
-

Overview

This project demonstrates a complete cloud-native deployment pipeline, taking the **Trendify** application from source code to a monitored, internet-facing deployment on Kubernetes.

The pipeline covers:

- Source control with **GitHub**
- Containerization with **Docker**, published to **Docker Hub**
- Infrastructure as Code with **Terraform**
- Container orchestration on **Amazon EKS**
- Public exposure via an **AWS Load Balancer**
- CI/CD automation with **Jenkins**
- Observability with **Prometheus**, **Grafana**, and **Metrics Server**

Tech Stack

Category	Tools / Services
Source Control	GitHub
CI/CD	Jenkins
Containerization	Docker, Docker Hub
Infrastructure as Code	Terraform
Orchestration	Amazon EKS, Kubernetes
Networking	AWS Load Balancer, VPC, Subnets, IGW
Monitoring	Prometheus, Grafana, Metrics Server, Helm
Cloud Provider	AWS (region: ap-south-1)

Architecture

Developer

|

v

GitHub

|

v

Jenkins

|

+-----> Docker Build

|

+-----> Docker Hub

|

v

Amazon EKS

|

+-----> Kubernetes Deployment

|

v

Trendify Pods

|

+-----> LoadBalancer

|

v

Trendify Application

Monitoring:

EKS

|

+-----> Metrics Server

|

+-----> Prometheus

|

+-----> Grafana

Project Structure

Trend/

```
├── Jenkinsfile
├── Dockerfile
├── dist/
├── kubernetes/
│   ├── deployment.yaml
│   └── service.yaml
├── terraform/
│   ├── eks.tf
│   ├── jenkins.tf
│   ├── eks_iam.tf
│   ├── node_group.tf
│   ├── provider.tf
│   ├── vpc.tf
│   ├── subnets.tf
│   ├── route_table.tf
│   └── internet_gateway.tf
├── .gitignore
└── README.md
```

Prerequisites

- AWS account with IAM permissions for EKS, EC2, VPC, and IAM
- [Terraform](#)
- [Docker](#)
- [kubectl](#)
- [Helm](#)
- [AWS CLI](#) configured with appropriate credentials
- Jenkins server (provisioned via Terraform, see below)
- Docker Hub account

Setup & Deployment Guide

1. Clone the Repository

```
git clone https://github.com/Akshaya33-Devops/CloudNative-App-Deployment.git
cd CloudNative-App-Deployment/Trend
```

2. Containerize the Application

The Trendify application ships as a production-ready Vite dist/ build, served with Nginx.

Dockerfile

```
FROM nginx:alpine
```

```
COPY dist/ /usr/share/nginx/html/
```

```
EXPOSE 80
```

Build and test locally:

```
docker build -t akshayanimuthu/cloudnative-trend:dev .
docker run -d -p 8081:80 akshayanimuthu/cloudnative-trend:dev
```

Visit <http://localhost:8081> to verify the container serves the app correctly.

In production, the image is not pushed manually — it is built and published to Docker Hub automatically by the Jenkins CI/CD pipeline (see [CI/CD Pipeline](#)):

Docker Hub repository: akshayanimuthu/cloudnative-trend
Tags: dev, <jenkins-build-number>

3. Provision AWS Infrastructure with Terraform

Terraform provisions the full networking and compute stack in ap-south-1:

```
VPC (10.0.0.0/16)
├── Public Subnet 1 (10.0.1.0/24)
├── Public Subnet 2 (10.0.2.0/24)
├── Internet Gateway
├── Route Table
├── EKS Cluster
├── EKS Node Group (t3.small, min 1 / desired 2 / max 3)
└── Jenkins EC2 instance
```

```
cd terraform
terraform init
terraform plan
terraform apply
```

The EKS cluster (cloudnative-eks) is created with authentication mode API_AND_CONFIG_MAP so Jenkins can be granted cluster access.

4. Configure Jenkins

Terraform provisions a Jenkins EC2 instance pre-installed with:

- Jenkins
- Git
- Docker
- AWS CLI
- kubectl

Jenkins authenticates to AWS via an **IAM role attached to the EC2 instance** (cloudnative-jenkins-role) — no static AWS access keys are stored in Jenkins.

EKS access for this role is granted using an aws_eks_access_entry with the AmazonEKSAAdminPolicy, giving Jenkins permission to run kubectl commands against the cluster.

5. Deploy to Amazon EKS

Kubernetes Deployment (kubernetes/deployment.yaml)

- 2 replicas
- Image: akshayamanimuthu/cloudnative-trend

Kubernetes Service (kubernetes/service.yaml)

- Type: LoadBalancer
- Port: 80 → Target Port: 80

Apply manually if needed:

```
aws eks update-kubeconfig --region ap-south-1 --name cloudnative-eks
```

```
kubectl apply -f kubernetes/deployment.yaml
```

```
kubectl apply -f kubernetes/service.yaml
```

6. CI/CD Pipeline

The Jenkinsfile defines the following pipeline stages:

```
Checkout
↓
Docker Build
↓
Docker Push
↓
Configure EKS
↓
```

Deploy to EKS



Verify Deployment

Stage	What it does
Checkout	Pulls main branch from GitHub
Docker Build	Builds akshayanimuthu/cloudnative-trend:\${BUILD_NUMBER} and :dev
Docker Push	Pushes both tags to Docker Hub using the dockerhub-creds credential
Configure EKS	Runs <code>aws eks update-kubeconfig</code> to point <code>kubectl</code> at the cluster
Deploy to EKS	Applies manifests and updates the deployment image via <code>kubectl set image</code>
Verify Deployment	Checks rollout status, pods, and service

Key verification commands:

```
kubectl rollout status deployment/trendify
kubectl get pods
kubectl get service trendify-service
```

A successful pipeline run results in Trendify pods reporting 2/2 Running.

7. Monitoring

Install Helm and add the Prometheus community chart repository:

```
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm repo update
```

Create a dedicated namespace and install the kube-prometheus-stack:

```
kubectl create namespace monitoring
```

```
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

The kube-prometheus-stack installs Prometheus, Grafana, Alertmanager, Prometheus Operator, Node Exporter, and kube-state-metrics.

Metrics Server is installed separately and provides Kubernetes resource metrics used by commands such as `kubectl top nodes`:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
kubectl top nodes
```

Accessing the Application

Kubernetes provisions an AWS Load Balancer automatically. The application was publicly reachable at the generated ELB DNS name, for example:

```
http://a3240ac9d8965468eb922d15501e3602-2147474683.ap-south-1.elb.amazonaws.com
```

Request flow:

Internet → AWS Load Balancer → Kubernetes Service → Trendify Pods → Nginx → Trendify Application

Accessing Monitoring Dashboards

Grafana

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

Open `http://localhost:3000` and log in with the admin credentials stored in the Kubernetes secret. Use the built-in Kubernetes dashboards to monitor cluster CPU/memory, node resources, namespace resources, and pod resources.

Prometheus

```
kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090
```

Open `http://localhost:9090` to verify that all scrape targets report UP, and run the up query to confirm healthy targets.

Cleanup

To avoid ongoing AWS costs, resources were torn down in this order:

1. Remove the monitoring stack

```
helm uninstall monitoring -n monitoring
kubectl delete namespace monitoring
```

2. Destroy the AWS infrastructure

```
terraform state list
terraform plan -destroy
terraform destroy
```

Verify cleanup:

```
terraform state list                                # should return
nothing
aws eks describe-cluster --name cloudnative-eks --region ap-south-1 #
should return ResourceNotFoundException
```

An empty result from `terraform state list` confirms Terraform has no remaining managed resources.

Project State

Preserved:

- GitHub repository
- Docker Hub repository / image
- Local project files
- Terraform configuration
- Kubernetes manifests
- Jenkinsfile
- README.md

Removed (to avoid cost):

- EKS cluster and worker nodes
- Jenkins EC2 instance
- VPC and associated networking resources
- Monitoring stack
- AWS Load Balancer

The AWS infrastructure was destroyed after project completion to avoid ongoing costs. The application and monitoring dashboards are **not currently running**. The Terraform, Kubernetes, Docker, and Jenkins configuration remains in the repository and can be used to recreate the deployment.

Screenshots

Screenshots covering the GitHub repository, Dockerfile, Docker Hub image, Jenkins credentials and pipeline, Terraform infrastructure, EKS cluster, Kubernetes deployment/pods/service, monitoring, Grafana and Prometheus dashboards, and the live application are available in the repository's `screenshots/` folder.

```
### GitHub Repository
![GitHub Repository] (screenshots/github-repository.png)

### Dockerfile
![Dockerfile] (screenshots/dockerfile.png)

### Docker Hub Image
![Docker Hub Image] (screenshots/dockerhub-image.png)

### Jenkins Credentials
![Jenkins Credentials] (screenshots/jenkins-credentials.png)

### Jenkins Pipeline
![Jenkins Pipeline] (screenshots/jenkins-pipeline.png)
```

Terraform Infrastructure
![Terraform Infrastructure](screenshots/terraform-infrastructure.png)

EKS Cluster
![EKS Cluster](screenshots/eks-cluster.png)

Kubernetes Deployment
![Kubernetes Deployment](screenshots/kubernetes-deployment.png)

Kubernetes Pods
![Kubernetes Pods](screenshots/kubernetes-pods.png)

Kubernetes Service
![Kubernetes Service](screenshots/kubernetes-service.png)

Kubernetes Monitoring
![Kubernetes Monitoring](screenshots/kubernetes-monitoring.png)

Grafana Dashboard Overview
![Grafana Dashboard Overview](screenshots/grafana-dashboard-overview.png)

Grafana Dashboard
![Grafana Dashboard](screenshots/grafana-dashboard.png)

Live Trendify Application
![Live Trendify Application](screenshots/live-application.png)

Prometheus Up Query
![Prometheus Up Query](screenshots/prometheus-up-query.png)

Author

Akshaya

- GitHub: [Akshaya33-Devops](#)
- Docker Hub: [akshayanimuthu](#)